

MAML-Select: An Online Adaptive Client Selection Method for Federated Learning via Meta-Learning

Abstract—In Federated Learning (FL), choosing clients for updating the shared model in each round is a combinatorial, resource constraints, and NP-hard scheduling problem. Clients differ in data utility, training speed, battery capacity and bandwidth. Since only a small cohort can communicate per round, conventional client-selection strategies often favor clients with high training utility or fast updates, which can repeatedly select a narrow group of clients, leaving slower or temporarily low-utility clients underused, reducing participation coverage, and making training process less efficient. Existing methods like FedCS and TiFL use system state to reduce straggler effects, whereas Oort, FedCor, CriticalFL, and FedGCS score clients using training utility, loss correlation, critical-period timing, or learned client combinations. However, these methods often rely on a fixed scoring rule, or update it slowly, even though a client’s usefulness changes as the global model evolves. Model-Agnostic Meta-Learning Select (MAML-Select) removes this fixed-rule assumption, and frames each round as a fast adaptation task. A lightweight policy network is learned using the previous round’s observed loss reduction and latency, and is used for ranking clients. After the selected clients return their updates, the same policy is meta-updated using the new feedback. We evaluate MAML-Select on Fashion-MNIST, CIFAR-10, and CIFAR-100 with 100 clients, $K = 10$ selected clients per round, a Dirichlet non-IID split with $\alpha = 0.5$, and three random seeds. On Fashion-MNIST, CIFAR-10, and CIFAR-100, MAML-Select provides competitive accuracy with significantly lower cumulative Tera Floating-Point Operations (TFLOPs) and modelled energy consumption. Higher values of Jain-index values also demonstrates the fairness of the proposed method.

Impact Statement—FL is central to privacy-preserving AI across heterogeneous environments, such as mobile health, intelligent transportation, industrial sensing, and smart-city systems, but these deployments often face tight computational-capacity, energy-consumption, and latency budgets. MAML-Select contributes an adaptive selection mechanism that reduces unnecessary client-training computational cost while preserving broad participation coverage. By using recent training feedback to re-rank clients, the method supports FL systems that respond to changing device conditions instead of relying on a fixed rule. The reported energy-consumption values are modelled estimates that follow the declared simulator and device-tier setup, giving a transparent basis for comparing selection policies. The broader impact is a more resource-aware path for deploying distributed AI at the edge, including the industrial Internet of Things (IIoT), especially when model updates must remain efficient, reproducible, and privacy-preserving.

Index Terms—Federated learning, client selection, meta-learning, and computational efficiency.

I. INTRODUCTION

FEDERATED Learning (FL) trains a shared model across many devices without moving their raw data to the server. [1]. Clients train models locally and share only model updates. However, they differ in hardware, network conditions, and local data. Their heterogeneity creates statistical drift under non-independent and identically distributed (non-IID)

data and straggler delays under synchronous training [2]–[6]. Despite these advances, the problem of *whom to select* each round remains open.

Client selection chooses a subset $\mathcal{S}_t \subset \{1, \dots, N\}$ each round. Client utility is non-stationary: a useful client can become redundant later, while an overlooked client can become important in the long run. System-aware methods improve client participation, however, selection bias avoids the fairness among clients as well as slow yet data-rich clients [7], [8]. Utility-aware methods prioritize information gain, but may incur high latency or expensive server-side modelling [9]–[13]. Learning-based selectors use histories, features, or reinforcement signals [14]–[16]. An efficient round-by-round mechanism for adapting the ranking policy to evolving utility remains underexplored.

We propose MAML-Select, a client-selection framework based on a Model-Agnostic Meta-Learning (MAML)-style fast adaptation step [17], which learns the policy network via recent feedback before the next ranking decision. Unlike reinforcement-learning (RL)-based or contextual-bandit selectors, the proposed MAML-Select uses rapid gradient-based policy adaptation. Unlike personalization of FL client models [18], the proposed MAML adapts the selector, rather than the client model. The proposed approach is useful when devices have limited computational capacity, strict deadlines, or intermittent connectivity. The major contributions can be summarized as follows.

- 1) We formulate the FL client selection problem as a meta-learning task and introduce the MAML-Select algorithm with rapid gradient-based adaptation for online client selection.
- 2) Experiments on Fashion-MNIST, CIFAR-10, and CIFAR-100 under non-IID settings ($\alpha = 0.5$, three seeds) show that MAML-Select delivers accuracy competitive with FedAvg and FedGCS at lower computational cost, and higher accuracy than FedCS, Oort, TiFL, and FedCor; sensitivity and ablation studies over λ , the inner-loop steps, the selector width, and the state features support the design choices.
- 3) We demonstrate that MAML-Select *lowers cumulative training TFLOPs* by up to 21.5% and modelled energy consumption by up to 24.7% relative to FedAvg, with paired-test support for the resource reductions.

Organization: The rest of this paper is organized as follows. Section II reviews related work on client selection in FL. Section III details the system model and problem formulation. The proposed MAML-Select algorithm is described in Section IV, followed by evaluation and results in Section V. Section VI concludes this paper and discusses future research directions.

II. RELATED WORK

In the following, the classification of existing client selection methods for the FL algorithm are presented.

1) *System-Aware Selection*: System-aware selection addresses stragglers. FedCS selects feasible clients under resource constraints, while TiFL groups devices into online performance tiers [7], [8]. Recent studies show similar concerns for slow but data rich clients in resource-constrained medical IoT and asynchronous FL with heterogeneous objectives [4], [5]. MAML-Select uses feedback from the previous training phase, and learns the selection policy based on six parameters.

2) *Data-Aware and Utility-Based Selection*: Data-aware methods select clients for model utility rather than only speed. Oort prioritizes data-rich and fast clients, and FedCor models correlations between client losses [9], [10]. CriticalFL uses critical learning periods, while FedGCS searches for client combinations in a learned continuous space [12], [13]. Other work couples selection with aggregated design [11], [19]. MAML-Select instead targets rapid adaptation of the ranking policy as client utility evolves.

3) *Learning-Based and Meta-Learning Approaches*: Learning-based selectors improve from observed feedback. FAVOR uses a deep Q-network (DQN), while neural contextual bandits learn from client features and rewards [14], [15]. Recent work also studies deep-RL selection for robustness and fairness-aware inclusive participation [16], [20]. In [18], MAML is employed to personalize client FL models; however, in this work, MAML-Select adapts the server-side ranking policy, rather than the FL models.

Table I consolidates these distinctions. For methods without a stated closed-form selection complexity, it reports the dominant server-side operation.

Difference from prior work: MAML-Select meta-learns the selection policy h_ϕ : unlike heuristic, RL, contextual-bandit, and generative selectors, it uses a MAML inner update to convert recent feedback into fast policy weights before the next cohort is scored. A fixed heuristic or bandit may update rewards over time, but does not adapt the scoring-network parameters from a support set immediately before selection.

III. SYSTEM MODEL AND PROBLEM FORMULATION

We consider a central server and N heterogeneous clients indexed by $\mathcal{N} = \{1, 2, \dots, N\}$. Client i stores a local dataset $\mathcal{D}_i$ with n_i samples. The global objective is to minimize the empirical risk over the distributed data as

$$\min_{\theta \in \mathbb{R}^d} F(\theta) = \sum_{i=1}^N \frac{n_i}{n} F_i(\theta), \quad (1)$$

where $\theta \in \mathbb{R}^d$ denotes the global model, $n = \sum_{i=1}^N n_i$, and $F_i(\theta) = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell(\theta; x_{i,j}, y_{i,j})$ is the local empirical risk, with $\ell(\cdot)$ the per-sample loss (e.g. cross-entropy).

At round t , let $\mathcal{A}_t \subseteq \mathcal{N}$ denote the available clients and $\mathcal{S}_t \subseteq \mathcal{A}_t$ the selected cohort. The server maintains the global model and selection policy; clients keep data local and return only model updates, loss summaries, and resource-state feedback. Selection is thus a server-side decision before each broadcast, with only a small fraction of clients training per round.

Client i has per-round latency $T_{i,t} = T_{i,t}^{\text{comp}} + T_{i,t}^{\text{comm}}$, split into computation and communication. The computation time is $T_{i,t}^{\text{comp}} = E n_i C_{\text{model}} / f_{i,t}$, where $E n_i C_{\text{model}}$ is the total local-training work over E epochs and n_i samples, C_{model} is the per-sample computational cost (forward-plus-backward FLOPs), and $f_{i,t}$ is the device's effective processing rate in FLOPs/s, set by its hardware tier. The communication time is $T_{i,t}^{\text{comm}} = M / B_{i,t} + \delta_{i,t}$, where M is the size of the uploaded model update, $B_{i,t}$ is the uplink bandwidth, and $\delta_{i,t}$ is a fixed propagation/queuing delay [7]. In synchronous FL, the slowest selected client determines the round latency:

$$T_t^{\text{round}} = \max_{i \in \mathcal{S}_t} T_{i,t}. \quad (2)$$

Let $a_{i,t}$ indicate whether client i is selected. Under policy π_ϕ , the constrained client-selection problem is

$$\begin{aligned} \min_{\pi_\phi} \quad & \sum_{t=1}^T [F(\theta_{t+1}) + \lambda T_t^{\text{round}}] \\ \text{s.t.} \quad & \sum_{i=1}^N a_{i,t} = K, \quad a_{i,t} \in \{0, 1\}, \\ & a_{i,t} \leq \mathbf{1}\{i \in \mathcal{A}_t\}, \end{aligned} \quad (3)$$

where K is the cohort size, T is the number of rounds, and $\lambda \geq 0$ controls the latency-loss trade-off, and the client selection subset is updated as $\mathcal{S}_t = \{i : a_{i,t} = 1\}$. A larger λ places more emphasis on latency, while $\lambda = 0$ yields loss-only selection. We assume $|\mathcal{A}_t| \geq K$. The model θ_{t+1} results from local training and aggregation over $\mathcal{S}_t$. The problem remains challenging because the action space is combinatorial and client utility evolves during training.

IV. MAML-SELECT ALGORITHM

Building on (3), MAML-Select treats client selection as a meta-learning problem: client utility changes as training progresses. In the following, algorithmic details of selection policy are given.

1) *State Space*: The server constructs a state vector $\mathbf{s}_{i,t} \in \mathbb{R}^6$ for each client i at round t . It contains local training loss $\hat{L}_{i,t}$, gradient norm $\|\nabla F_i\|_2$, estimated latency $\hat{T}_{i,t}$, battery level $b_{i,t}$, selection frequency $q_{i,t}$, and staleness $\tau_{i,t}$ (rounds since last selected) [11], [15]. The state vector $\mathbf{s}_{i,t}$ has six entries: Statistical utility ($\hat{L}_{i,t}$, $\|\nabla F_i\|_2$), System feasibility ($\hat{T}_{i,t}$, $b_{i,t}$) and Participation balance ($q_{i,t}$, $\tau_{i,t}$). The local training loss $\hat{L}_{i,t}$ tells us if the client still has useful gradients. The gradient norm $\|\nabla F_i\|_2$ tells us the size and value of the update. The estimated latency $\hat{T}_{i,t}$ tells us the straggler and round-time risk. The battery level $b_{i,t}$ tells us the energy feasibility and the drop-out risk. The selection frequency $q_{i,t}$ helps avoid over-using a few clients. The staleness $\tau_{i,t}$, the rounds since last selected, helps avoid starving clients. Each round t is a task $\mathcal{T}_t$: rank clients so that the top- K cohort minimizes the per-round utility.

MAML-Select trains a meta-policy network $h_\phi(\mathbf{s}_{i,t})$, parameterized by ϕ , that learns an adaptable policy to the current round using data from the preceding round. The policy is a 6-64-64-1 multilayer perceptron (MLP) with ReLU activations and 4,673 trainable parameters. The support set represents the

TABLE I
COMPARISON OF REPRESENTATIVE METHODS AND MAML-SELECT.

Method	Primary objective	Adaptation mechanism	Selection complexity or dominant operation	Optimization strategy
FedCS [7]	Increase feasible updates under resource constraints	Resource estimates before selection	Resource-constrained subset search	Deadline-aware scheduling
TiFL [8]	Mitigate stragglers while preserving accuracy	Online tier updates	Tiering and probabilistic sampling	Adaptive tier-based selection
Oort [9]	Improve time-to-accuracy	Utility and speed feedback	Utility scoring and guided sampling	Exploration and exploitation
FedCor [10]	Accelerate convergence under non-IID data	Loss-correlation model	Gaussian process (GP) covariance operations	GP-based active selection
FAVOR [14]	Counterbalance non-IID bias	Experience-driven policy updates	DQN inference and training	Deep Q-learning
NCCB [15]	Learn contextual client combinations	Client features and reward feedback	Locality-sensitive hashing (LSH) feature extraction and bandit updates	Neural contextual combinatorial bandit
FedCG [11]	Couple selection and gradient compression	Capability-aware compression updates	Submodular optimization and linear programming	Joint iterative optimization
CriticalFL [12]	Exploit critical learning periods	Period-aware participation	Base-selector dependent	Selector augmentation
FedGCS [13]	Optimize client combinations	Search in a learned continuous space	Latent-space optimization and beam search	Gradient-based generative selection
Kazemi et al. [16]	Improve robustness under poisoning attacks	Experience-driven policy updates	Deep-RL inference and training	Deep reinforcement learning
Vox Populi [20]	Promote fair and inclusive participation	Fairness-aware client selection	Framework-dependent	Inclusive selection framework
Per-FedAvg [18]	Personalize the client model to local data	MAML adaptation of the client model, not the selector	No client selection; per-client model fine-tuning	Model-level meta-learning
MAML-Select	Adapt rankings under evolving client utility	Fast adaptation from recent round feedback	$\mathcal{O}(N \phi)$ scoring plus a gradient-adaptation step	Gradient-based meta-learning

past experience: $\mathcal{D}_{sup}(t) = \{(\mathbf{s}_{i,t-1}, c_{i,t-1}) \mid i \in \mathcal{S}_{t-1}\}$, containing client states and observed costs from round $t-1$. The candidate set represents the current decision: $\mathcal{D}_{cand}(t) = \{\mathbf{s}_{i,t} \mid i \in \mathcal{A}_t\}$, containing the states of available clients.

2) *Utility Function*: The per-client utility $c_{i,t}$ supplies feedback to the ranking policy. Consistent with (3), the utility of the client i is defined as

$$c_{i,t} = \underbrace{\lambda \max(0, T_{i,t} - T_{target})}_{\text{latency penalty}} - \underbrace{\Delta_{i,t}}_{\text{local loss reduction}}, \quad (4)$$

where $\Delta_{i,t} = \hat{L}_{i,t}^{before} - \hat{L}_{i,t}^{after}$ is the local loss reduction for selected client i ($\hat{L}_{i,t}^{before}$ before and $\hat{L}_{i,t}^{after}$ after its E local epochs), thus $\Delta_{i,t} > 0$ marks a useful update. The target T_{target} is a soft deadline, and minimizing $c_{i,t}$ favors lower latency and larger loss reduction.

A. The MAML-Select Algorithm

The algorithm proceeds in three phases per round:

Phase 1 (Inner Loop Adaptation (Look-Ahead)): At the start of round t , the server adapts the meta-policy ϕ_t on $\mathcal{D}_{sup}(t)$ using one gradient-descent step:

$$\phi'_t = \phi_t - \beta \nabla_{\phi_t} \mathcal{L}_{sup}(h_{\phi_t}; \mathcal{D}_{sup}(t)), \quad (5)$$

where $\mathcal{L}_{sup}$ is the mean-squared-error (MSE) loss between predicted scores and observed costs from round $t-1$, and β is the inner learning rate.

Lemma 1 (Inner-step descent). *Let the support loss $g_t(\phi) = \mathcal{L}_{sup}(h_{\phi}; \mathcal{D}_{sup}(t))$ be L -smooth, that is, ∇g_t is L -Lipschitz. If the inner step size obeys $0 < \beta < 2/L$, the inner update (5) is a descent step on the support loss:*

$$g_t(\phi'_t) \leq g_t(\phi_t) - \beta \left(1 - \frac{L\beta}{2}\right) \|\nabla g_t(\phi_t)\|_2^2.$$

Proof. By L -smoothness, g_t obeys the descent lemma

$$g_t(\psi) \leq g_t(\phi) + \nabla g_t(\phi)^\top (\psi - \phi) + \frac{L}{2} \|\psi - \phi\|_2^2.$$

Substituting $\psi = \phi'_t$, $\phi = \phi_t$, and the inner update $\phi'_t - \phi_t = -\beta \nabla g_t(\phi_t)$ yields the stated bound, whose coefficient

$\beta(1 - L\beta/2)$ is positive precisely for $0 < \beta < 2/L$; the step therefore never increases g_t and strictly decreases it unless $\nabla g_t(\phi_t) = 0$.

A single support-set step therefore reduces the round's ranking loss, justifying adaptation before the current candidates are scored.

Phase 2 (Client Selection): The adapted policy $h_{\phi'_t}$ scores the available clients and selects the K clients with the lowest predicted costs:

$$\mathcal{S}_t = \text{Top-}K_{i \in \mathcal{A}_t} (-h_{\phi'_t}(\mathbf{s}_{i,t})). \quad (6)$$

Phase 3 (FL Execution and Outer Loop Update): The selected clients perform standard FL training. Their observed costs form $\mathcal{D}_{query}(t) = \{(\mathbf{s}_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$. The meta-policy is updated on this query set:

$$\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi'_t} \mathcal{L}_{query}(h_{\phi'_t}; \mathcal{D}_{query}(t)), \quad (7)$$

where η is the meta-learning rate. Query-loss gradient is evaluated at the adapted weights ϕ'_t and applied to the meta-parameters ϕ_t , which omits the second-order derivatives. The full procedure is detailed in Algorithm 1 and Fig. 1.

Lemma 2 (Outer-step descent). *Let the query loss $q_t(\phi) = \mathcal{L}_{query}(h_{\phi}; \mathcal{D}_{query}(t))$ be L_q -smooth, and let*

$$\delta_t := \|\nabla q_t(\phi'_t) - \nabla q_t(\phi_t)\|_2$$

denote the first-order approximation error, the gap between the first-order MAML direction $\nabla q_t(\phi'_t)$, evaluated at the adapted weights ϕ'_t , and the exact gradient $\nabla q_t(\phi_t)$. If the outer step size obeys $0 < \eta \leq 1/L_q$, the meta-update (7) satisfies

$$q_t(\phi_{t+1}) \leq q_t(\phi_t) - \frac{\eta}{2} \|\nabla q_t(\phi_t)\|_2^2 + \frac{\eta}{2} \delta_t^2,$$

with the inner step controlling this error through $\delta_t \leq L_q \beta \|\nabla g_t(\phi_t)\|_2$.

Proof. Write $\phi_{t+1} = \phi_t - \eta d_t$ with $d_t := \nabla q_t(\phi'_t)$. With $\eta \leq 1/L_q$, the descent lemma for q_t gives

$$q_t(\phi_{t+1}) \leq q_t(\phi_t) - \eta \nabla q_t(\phi_t)^\top d_t + \frac{\eta}{2} \|d_t\|_2^2.$$

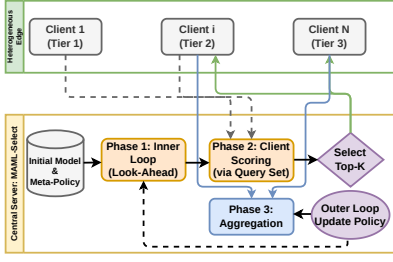


Fig. 1. System model and MAML-Select workflow. A central server observes client states, selects K clients from the available pool, aggregates their local updates, and uses returned local-loss and system feedback for support-set adaptation, candidate scoring, and query-set meta-update.

Completing the square, with $-a^\top b + \frac{1}{2}\|b\|_2^2 = \frac{1}{2}\|b - a\|_2^2 - \frac{1}{2}\|a\|_2^2$ for $a = \nabla_{q_t}(\phi_t)$ and $b = d_t$, rewrites the right-hand side as

$$q_t(\phi_t) - \frac{\eta}{2}\|\nabla_{q_t}(\phi_t)\|_2^2 + \frac{\eta}{2}\|d_t - \nabla_{q_t}(\phi_t)\|_2^2.$$

Since $\|d_t - \nabla_{q_t}(\phi_t)\|_2 = \delta_t$, the first bound follows; the second uses L_q -smoothness and (5), giving $\delta_t \leq L_q\|\phi'_t - \phi_t\|_2 = L_q\beta\|\nabla_{g_t}(\phi_t)\|_2$.

Since the bound makes δ_t proportional to β for a bounded support-gradient norm, the error term $\frac{\eta}{2}\delta_t^2 = \mathcal{O}(\beta^2)$ is negligible at $\beta = 10^{-2}$, and the first-order meta-update behaves like exact gradient descent on the query loss. Over a run, this per-round descent yields a convergence rate.

Corollary 1 (Approximate stationarity). *Suppose, in addition, that the query objective is stationary across rounds, $q_t \equiv q$ with $q^* = \inf_{\phi} q(\phi) > -\infty$. Then the meta-updates (7) satisfy*

$$\min_{1 \leq t \leq T} \|\nabla q(\phi_t)\|_2^2 \leq \frac{2(q(\phi_1) - q^*)}{\eta T} + \frac{1}{T} \sum_{t=1}^T \delta_t^2.$$

With $\delta_t \leq L_q\beta\|\nabla_{g_t}(\phi_t)\|_2$, the policy reaches an $\mathcal{O}(\beta^2)$ -approximate stationary point of q at the rate $\mathcal{O}(1/(\eta T))$.

Proof. Rearranging Lemma 2 as $\frac{\eta}{2}\|\nabla q(\phi_t)\|_2^2 \leq q(\phi_t) - q(\phi_{t+1}) + \frac{\eta}{2}\delta_t^2$ and summing over $t = 1, \dots, T$ telescopes the function values to

$$\frac{\eta}{2} \sum_{t=1}^T \|\nabla q(\phi_t)\|_2^2 \leq q(\phi_1) - q^* + \frac{\eta}{2} \sum_{t=1}^T \delta_t^2.$$

Dividing by $\eta T/2$ and lower-bounding the average by its minimum gives the inequality.

Lemmas 1 and 2 give one-step descent for the inner and outer updates, and Corollary 1 converts the outer guarantee into an $\mathcal{O}(\beta^2)$ -stationary convergence rate. These trends are visible throughout training: Fig. 2(b) shows the meta-update magnitude $\|\phi_{t+1} - \phi_t\|_2$ on CIFAR-100 shrinking and settling as the run proceeds, the empirical signature of the $\mathcal{O}(\beta^2)$ -stationary point predicted by Corollary 1.

MAML-Select differs from fixed-rule selectors because its ranking policy is adapted before each decision. It also differs from RL selectors because the update is an explicit gradient step on recent support feedback. The method is designed for rapidly changing client utility.

Algorithm 1 MAML-Select Algorithm

```

1: Initialize: Global model  $\theta_0$ ; meta-policy parameters  $\phi$ ; inner LR  $\beta$ ; outer LR  $\eta$ ; trade-off  $\lambda$ .
2: for round  $t = 1, \dots, T$  do
3:   Server broadcasts global model  $\theta_t$  to all clients.
4:   Collect state vectors  $s_{i,t}$  from available clients.
5:   if  $t > 1$  then
6:     // Phase 1: Inner Loop Adaptation
7:     Construct  $\mathcal{D}_{sup}$  from round  $t-1$  results.
8:     Compute fast weights  $\phi'_t$  using (5).
9:   else
10:     $\phi'_t \leftarrow \phi_t$  // No adaptation for the first round
11:   end if
12:   // Phase 2: Client Selection
13:   Score clients:  $\text{score}_i = h_{\phi'_t}(s_{i,t})$  for all  $i \in \mathcal{A}_t$ .
14:   Select lowest-cost clients:  $\mathcal{S}_t \leftarrow \text{Top-}K(-\text{scores})$ .
15:   // FL Training Round
16:   Receive local updates  $\{\Delta\theta_{i,t}\}_{i \in \mathcal{S}_t}$ , local losses, and system metrics.
17:   Aggregate:  $\theta_{t+1} \leftarrow \theta_t + \sum_{i \in \mathcal{S}_t} \frac{n_i}{\sum_{j \in \mathcal{S}_t} n_j} \Delta\theta_{i,t}$ .
18:   // Phase 3: Outer Loop Meta-Update
19:   Compute costs  $c_{i,t}$  using (4) for  $i \in \mathcal{S}_t$ .
20:   Construct  $\mathcal{D}_{query}(t) = \{(s_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$ .
21:   Meta-update:  $\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi'_t} \mathcal{L}_{query}(h_{\phi'_t}; \mathcal{D}_{query}(t))$ .
22: end for

```

1) *Computational Complexity:* Let $P = |\phi|$ denote the number of meta-policy parameters. For the 6-64-64-1 MLP used here, $P = 4,673$. A forward pass for one client costs $\mathcal{O}(P)$, so scoring N available clients costs $\mathcal{O}(NP)$. Selecting the best K clients can be implemented with a size- K heap in $\mathcal{O}(N \log K)$, or by partial sorting. The inner support update and outer query update each use at most the selected cohort and therefore cost $\mathcal{O}(KP)$. Thus the per-round server-side selection computational cost is

$$\mathcal{O}(NP + N \log K + 2KP). \quad (8)$$

Since P and K are fixed in the experimental protocol ($P = 4,673$, $K = 10$), MAML-Select scales linearly with the client pool size N . The memory cost is $\mathcal{O}(P + N + K)$ for the policy parameters, client scores, and selected cohort. This results in contrast with correlation-matrix or Gaussian-Process selection approaches where the covariance operations can grow beyond linearly, and in common FedCor-style implementations cubically, with N [10].

V. EVALUATION AND RESULTS

We evaluate on three benchmarks: Fashion-MNIST with LightCNN-9 [21], and CIFAR-10 and CIFAR-100 with ResNet18 [22]. $N = 100$ clients are simulated with non-IID data partition (Dirichlet $\alpha = 0.5$), selecting $K = 10$ clients per round for $T = 200$ rounds (150 rounds for CIFAR-100) with $E = 5$ local epochs and batch size 32. Table II reports mean $\pm$ standard deviation over repeated runs. The tests use seeds 42, 123, and 2026. Local models use PyTorch default initialization and stochastic gradient descent (SGD) with learning rate 0.01, momentum 0.9, weight decay 10^{-4} , and a constant scheduler. The meta-policy uses one inner step with $\beta = 0.01$ and Adam for the outer update with $\eta = 0.001$. The selector seed is fixed to 2026 so the reported variance reflects model-training and data-partition randomness; the simulation is fully specified by Algorithm 1 and the stated parameters. Processing rates follow three device tiers that reflect the systems heterogeneity common in federated deployments [23]: Tier 1 (20%, 1.0 $\times$), Tier 2 (50%, 2.0 $\times$), and Tier 3 (30%, 4.0 $\times$). Resource accounting uses the same tiers: Tier 1 devices use 4 W power and 18 watt-hour (Wh) batteries, Tier 2 uses 7 W and 30 Wh, and Tier 3

uses 12 W and 48 Wh. Round time is straggler-bound by the slowest selected client. The target latency deadline T_{target} is set to the average round latency of Tier 2 clients, and the trade-off parameter $\lambda = 0.5$. Baselines are FedAvg, FedCS, Oort, TiFL, FedCor, FedGCS, and CriticalFL, as characterized in Table I.

A. Accuracy and Resource Cost

We estimate the client-training computational cost per round as

$$\text{TFLOPs}_{total} = \sum_{t=1}^T \sum_{i \in \mathcal{S}_t} E n_i C_{model} \times 10^{-12}, \quad (9)$$

where C_{model} is the same per-sample cost (FLOPs) as in $T_{i,t}^{comp}$, so the reported cost is consistent with the latency model (one ResNet18/CIFAR-10 round is ≈ 4.13 TFLOPs). Modelled energy accumulates per-tier power over local-training time:

$$E_{total} = \sum_{t=1}^T \sum_{i \in \mathcal{S}_t} \frac{P_i T_{i,t}^{comp}}{3600}, \quad (10)$$

where P_i is the device-tier power in watts and $T_{i,t}^{comp}$ is in seconds, so E_{total} accumulates in watt-hours (Wh).

Table II summarises all methods. MAML-Select attains accuracy on par with FedAvg and FedGCS on Fashion-MNIST (90.1% vs. 90.2% and 90.7%) and CIFAR-100 (58.2% vs. 59.7% and 58.7%). The proposed method also exceeds FedCS, Oort, TiFL, FedCor, and CriticalFL on CIFAR-100 accuracy. On CIFAR-10, a moderate accuracy reduction (75.6% vs. FedAvg 79.4%) can be observed for substantially lower computational cost. Overall, the MAML-Select offers a resource-aware operating point, staying close to the strongest accuracy baselines while reducing TFLOPs and modelled energy consumption across all reported non-IID settings. The meta-update computational overhead ($\varepsilon \approx 0.001$ GFLOPs per round) is negligible relative to client training.

Accuracy is also tested with paired t -tests over the three seeds. Paired tests against FedAvg support the resource reductions on all datasets: TFLOPs p -values are 0.009/0.002/0.012, and energy-consumption p -values are 0.007/0.001/0.003 for Fashion-MNIST/CIFAR-10/CIFAR-100. Against FedGCS, MAML-Select is statistically indistinguishable on CIFAR-10 ($p = 0.10$) and CIFAR-100 ($p = 0.49$), and trails by only 0.5 pp (percentage point) on Fashion-MNIST ($p = 0.04$). Against FedAvg, the two are statistically equivalent on Fashion-MNIST (-0.1 pp, $p = 0.84$), whereas CIFAR-10 (-3.8 pp, $p = 0.04$) and CIFAR-100 (-1.9 pp, $p = 0.04$) show the best energy efficiency at the expense of a negligible reduction in accuracy. For CIFAR-10, the 75.6% is the converged 200-round accuracy; the lower λ -study numbers are 100-round diagnostics. Lowering λ from 0.5 to 0.1 raises diagnostic accuracy by only about one point, so the residual gap is not mainly a λ effect but reflects the harder ResNet18 task, where one low-cost cohort per round can miss useful class variation. We therefore interpret the CIFAR-10 result as an efficiency-oriented operating point, where MAML-Select prioritizes lower resource use over peak final accuracy.

Fig. 2 summarises the CIFAR-100 behaviour. MAML-Select remains close to FedAvg and FedGCS on accuracy and loss

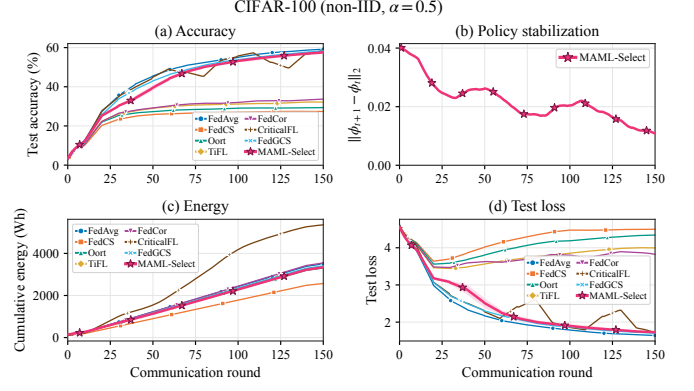


Fig. 2. CIFAR-100 convergence over 150 rounds. The panels report (a) accuracy, (b) the selector meta-update magnitude $\|\phi_{t+1} - \phi_t\|_2$ for MAML-Select (the empirical signature of Corollary 1), (c) cumulative modelled energy consumption, and (d) test loss.

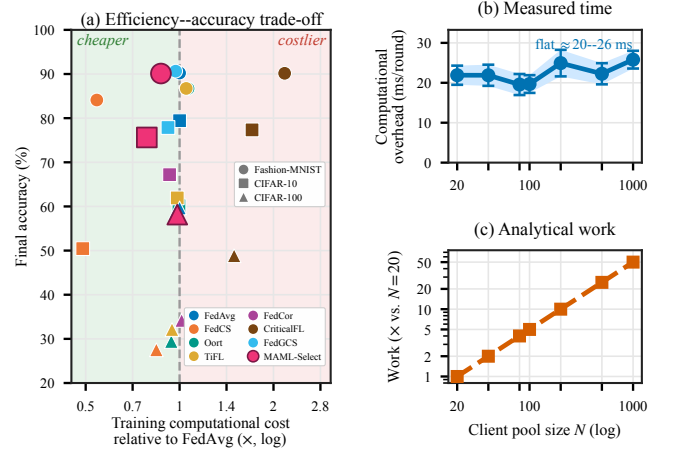


Fig. 3. (a) Efficiency-accuracy trade-off: the x -axis is cumulative training computational cost relative to FedAvg (log scale; $x < 1$ is cheaper than FedAvg), the y -axis is final accuracy, and marker shape encodes the dataset. (b) Measured per-round selection computational overhead (mean with 95% confidence intervals over 10 rounds) and (c) the analytical operation count $C(N, K) = NP + N \log K + 2KP$ from (8) (policy size $P = 4,673$, normalized to its value at $N = 20$), both as the client pool size N grows with the selection rate held at 10% ($K/N = 0.1$).

while staying in the low energy-consumption cluster, and panel (b) shows the selector meta-update settling.

Fig. 3(a) shows the efficiency-accuracy trade-off: MAML-Select is consistently in the lower-computational-cost region while staying close to the strongest baselines in accuracy. Relative to FedAvg over shared seeds, it lowers cumulative TFLOPs by 12.8%/21.5%/1.7% and modelled energy consumption by 16.4%/24.7%/4.7% on Fashion-MNIST/CIFAR-10/CIFAR-100, for accuracy changes of $-0.1/-3.8/-1.5$ percentage points. The main empirical gain is thus resource efficiency with competitive accuracy: the selector keeps broad data coverage while operating at a lower computational-cost point than several utility-driven alternatives.

We measure participation fairness with Jain's index on the per-client selection counts $\{p_i\}_{i=1}^N$, defined as $J = (\sum_i p_i)^2 / (N \sum_i p_i^2)$, which lies in $[1/N, 1]$ with $J = 1$ for perfectly even selection and smaller values for more concentrated selection. We also report participation coverage, the

TABLE II
BENCHMARK SUMMARY ACROSS NON-IID DATASETS. ACCURACY, PRECISION, RECALL, F1, AND COVERAGE ARE IN PERCENT; ENERGY IS IN WATT-HOUR (WH).

Method	Acc.	Prec.	Rec.	F1	TFLOPs	Energy	Jain	Cov.
<i>Fashion-MNIST</i>								
FedAvg	90.22±0.58	90.55±0.20	90.22±0.58	90.22±0.51	140±1	5752±72	0.95±0.00	100±0
FedCS	84.11±1.82	84.84±0.77	84.11±1.82	83.91±1.98	76±7	2772±211	0.10±0.00	10±0
Oort	86.70±0.19	86.66±0.21	86.70±0.19	86.42±0.26	149±8	5963±456	0.10±0.00	10±0
TiFL	86.73±0.34	86.62±0.26	86.73±0.34	86.51±0.34	147±6	6062±348	0.11±0.00	12±0
FedCor	89.35±0.50	89.31±0.51	89.35±0.50	89.22±0.57	121±5	4845±204	0.26±0.04	49±8
CriticalFL	90.16±0.47	90.24±0.38	90.17±0.47	90.11±0.42	304±143	12552±5860	0.98±0.01	100±0
FedGCS	90.65±0.29	90.73±0.16	90.65±0.29	90.63±0.26	136±4	5595±113	0.95±0.02	100±0
MAML-Select	90.11±0.47	90.43±0.04	90.11±0.47	90.04±0.30	122±2	4809±79	0.77±0.06	100±0
<i>CIFAR-10</i>								
FedAvg	79.43±1.84	80.01±1.28	79.43±1.84	79.36±1.78	8341±52	4798±24	0.95±0.00	100±0
FedCS	50.45±2.57	50.88±2.60	50.45±2.57	49.87±2.33	4081±600	2073±262	0.10±0.00	10±0
Oort	60.19±5.18	60.84±4.97	60.19±5.18	59.69±4.61	8303±1923	4677±856	0.10±0.00	10±0
TiFL	61.93±5.04	62.48±4.78	61.93±5.04	61.36±4.60	8200±2143	4800±1203	0.11±0.00	12±0
FedCor	67.20±5.33	67.83±4.74	67.20±5.33	66.81±5.37	7748±695	4452±435	0.20±0.02	31±5
CriticalFL	77.34±5.21	79.98±2.80	77.34±5.21	77.46±5.21	14239±4864	8205±2807	0.98±0.01	100±0
FedGCS	77.88±1.75	78.93±1.50	77.88±1.75	77.69±1.54	7656±278	4428±154	0.90±0.03	100±0
MAML-Select	75.63±1.43	76.51±1.55	75.63±1.43	75.25±1.34	6549±118	3610±60	0.61±0.01	100±0
<i>CIFAR-100</i>								
FedAvg	59.66±0.29	60.90±0.48	59.66±0.29	59.13±0.40	6331±6	3626±18	0.94±0.00	100±0
FedCS	27.52±0.29	27.93±0.10	27.52±0.29	26.39±0.09	5334±23	2659±12	0.10±0.00	10±0
Oort	29.41±1.73	29.82±1.77	29.41±1.73	28.13±1.73	5955±189	3421±181	0.10±0.00	10±0
TiFL	32.12±0.30	33.07±0.50	32.12±0.33	31.02±0.42	5986±28	3499±14	0.11±0.00	12±0
FedCor	34.18±0.24	33.89±0.26	34.18±0.60	32.93±0.20	6428±32	3668±18	0.12±0.00	14±0
CriticalFL	48.84±1.58	60.56±1.89	48.84±1.58	48.07±2.32	9469±392	5438±214	0.97±0.00	100±0
FedGCS	58.66±0.11	60.24±0.03	58.66±0.11	58.08±0.20	6140±24	3526±4	0.82±0.02	100±0
MAML-Select	58.15±0.51	59.87±0.55	58.15±0.51	57.66±0.33	6224±20	3457±6	0.78±0.03	100±0

percentage of clients selected at least once, $\text{Cov} = \frac{100}{N} |\{i : \sum_{t=1}^T a_{i,t} \geq 1\}|$, where $\text{Cov} = 100\%$ means no client is ever permanently excluded. MAML-Select reaches 100% client coverage on every dataset. While its Jain index (0.61–0.78) is lower than FedAvg and FedGCS, perfect coverage demonstrates that the policy does not permanently exclude any clients.

B. Selector Scaling

To validate the complexity expression in (8), we measured the server-side selection computational overhead of MAML-Select over 10 rounds. We varied the client pool size as $N \in \{20, 40, 80, 100, 200, 500, 1000\}$ and held the selection rate fixed at 10% ($K/N = 0.1$). Fig. 3(b) shows that the measured computational overhead stays nearly flat, between approximately 20 and 26 ms per round, even as N grows fifty-fold to 1000. With the selection rate held at 10%, the $\mathcal{O}(NP)$ scoring term dominates, so the operation count grows about linearly with N (Fig. 3(c)). The measured time stays flat because at these sizes the small $\mathcal{O}(NP)$ scoring cost is dominated by fixed per-round bookkeeping, so the selector is inexpensive even at $N = 1000$. The one-step inner loop is used intentionally. Additional inner steps increase the adaptation cost, while the ablation results show that one step gives the best accuracy with the lowest overhead.

C. Sensitivity and Ablation

These experiments test whether MAML-Select is stable when its design choices are perturbed. We probe four choices: the cost trade-off λ , the number of inner-loop steps, the selector width, and the six-feature policy state. To keep the diagnostic budget tractable, each ablation is reported on the dataset where it is most informative.

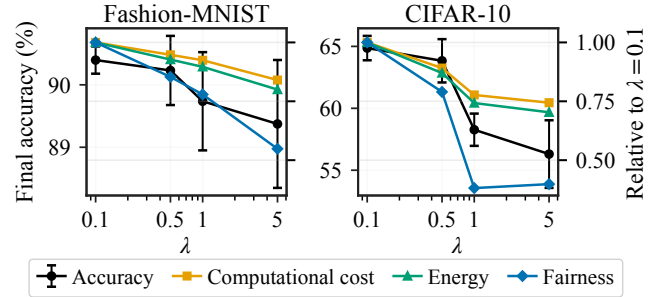


Fig. 4. Sensitivity to the cost trade-off λ on Fashion-MNIST (left) and CIFAR-10 (right). The black curve is accuracy. Computational cost, energy consumption, and fairness are normalized to their $\lambda = 0.1$ values. The trade-off is mild on Fashion-MNIST and sharper on CIFAR-10.

Fig. 4 reports the λ sweep on both Fashion-MNIST and CIFAR-10. A larger λ gives more weight to lower utility clients, so computational cost and energy consumption fall on both datasets. On Fashion-MNIST, accuracy stays nearly flat, within about one point. On CIFAR-10, the trade-off is sharper: accuracy drops from 64.9% at $\lambda = 0.1$ to 56.3% at $\lambda = 5$, while computational cost falls by about 26% and energy consumption by about 30%. Fairness also falls at large λ . We keep $\lambda = 0.5$ as the default. It stays close to the best accuracy, lowers cost, and avoids the sharp fairness loss seen at $\lambda \geq 1$.

Table III depicts the choices of two ablations. On CIFAR-10, a single inner step gives the highest accuracy (63.8%) and the lowest overhead (38.7 versus 45.2 ms at five steps), so we use one step. On Fashion-MNIST, accuracy stays within about 0.6 points as the width grows over $h \in \{16, 32, 64, 128\}$ (401 to 17,537 parameters), so the compact $h = 64$ default is robust.

TABLE III
INNER-STEP (CIFAR-10) AND SELECTOR-WIDTH (FASHION-MNIST)
ABLATIONS: FINAL ACCURACY AND SELECTION COMPUTATIONAL
OVERHEAD, MEAN $\pm$ STD OVER THREE SEEDS. DEFAULTS (1 STEP, $h = 64$)
IN BOLD.

Ablation	Setting	Acc. (%)	Computational overhead (ms)
Inner steps	1	63.8 $\pm$ 1.8	38.7
	2	63.1 $\pm$ 1.8	39.8
	5	62.2 $\pm$ 2.4	45.2
Width h	16	90.0 $\pm$ 0.3	10.2
	32	90.6 $\pm$ 0.3	10.4
	64	90.1 $\pm$ 0.5	10.2
	128	90.3 $\pm$ 0.7	11.6

Table IV removes one state feature at a time (Fashion-MNIST). Accuracy changes stay within about 0.35 points, so the policy is robust to any single removal. The largest drop comes from the battery feature (-0.33 pp), with latency and loss also contributing; removing the frequency feature raises accuracy slightly but lowers the Jain index the most ($0.78 \rightarrow 0.65$), so the participation features chiefly protect fairness. These results support the full six-feature state because it preserves both accuracy stability and participation balance.

TABLE IV
STATE-FEATURE ABLATION ON FASHION-MNIST ($n = 3$); EACH ROW
REMOVES ONE FEATURE FROM THE SIX-DIMENSIONAL STATE VECTOR.

Variant	Acc. (%)	TFLOPs	Jain
Full state vector	90.23 $\pm$ 0.55	122	0.78
w/o loss	90.16 $\pm$ 0.66	123	0.79
w/o gradient norm	90.30 $\pm$ 0.24	122	0.78
w/o latency	90.14 $\pm$ 0.14	119	0.70
w/o battery	89.90 $\pm$ 0.44	122	0.79
w/o frequency	90.47 $\pm$ 0.13	115	0.65
w/o staleness	90.37 $\pm$ 0.19	123	0.78

VI. CONCLUSION AND FUTURE WORK

We introduced MAML-Select, an online adaptive client-selection framework for heterogeneous FL that learns a lightweight ranking policy over both learning feedback and system-state information, adapting it with recent support-set feedback before each selection decision. Across Fashion-MNIST, CIFAR-10, and CIFAR-100, MAML-Select achieves competitive accuracy while reducing cumulative computational cost and modelled energy consumption compared with higher-cost selection strategies. The λ -sensitivity study shows that the cost trade-off can be tuned without destabilizing accuracy, and the ablations confirm that a single inner step, a compact policy network, and the full six-feature state are sufficient for stable selection. Future work will broaden the evaluation across heterogeneity levels, client-pool sizes, and naturally federated benchmarks, and will study an asynchronous extension that relaxes the synchronous-round assumption together with a differentially private treatment of the client-state vector so that selection does not leak device information.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [2] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5132–5143.
- [3] V. S. Mai, R. J. La, and T. Zhang, "A study of enhancing federated learning on non-IID data with server learning," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 11, pp. 5589–5604, 2024.
- [4] R. Tapwal, "Stragglers reimagined: Explainability-driven adaptive federated learning for resource constrained IoT system," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 2, pp. 1002–1011, 2026.
- [5] A. Forootani and R. Iervolino, "Asynchronous federated learning with nonconvex client objective functions and heterogeneous dataset," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 5, pp. 2610–2624, 2026.
- [6] K. Borazjani, P. Abdisarabshali, N. Khosravan, and S. Hosseinalipour, "Redefining non-IID data in federated learning for computer vision tasks: Migrating from labels to embeddings for task-specific data distributions," *IEEE Transactions on Artificial Intelligence*, 2026, early Access.
- [7] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *IEEE International Conference on Communications (ICC)*. IEEE, 2019, pp. 1–7.
- [8] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, and Y. Cheng, "TIFL: A tier-based federated learning system," in *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*, 2020, pp. 125–136.
- [9] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *15th USENIX Symposium on Operating Systems Design and Implementation*, 2021, pp. 19–35.
- [10] M. Tang, X. Ning, Y. Wang, J. Sun, Y. Wang, H. Li, and Y. Chen, "Fed-Cor: Correlation-based active client selection strategy for heterogeneous federated learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2022, pp. 10 102–10 111.
- [11] Y. Xu, Z. Jiang, H. Xu, Z. Wang, C. Qian, and C. Qiao, "Federated learning with client selection and gradient compression in heterogeneous edge systems," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5446–5461, 2024.
- [12] G. Yan, H. Wang, X. Yuan, and J. Li, "CriticalFL: A critical learning periods augmented client selection framework for efficient federated learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 5034–5044.
- [13] Z. Ning, C. Tian, M. Xiao, W. Fan, P. Wang, L. Li, P. Wang, and Y. Zhou, "FedGCS: A generative framework for efficient client selection in federated learning via gradient-based optimization," in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, 2024, pp. 4760–4768.
- [14] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *IEEE INFOCOM*. IEEE, 2020, pp. 1698–1707.
- [15] Q. Pan, H. Cao, Y. Zhu, J. Liu, and B. Li, "Contextual client selection for efficient federated learning over edge devices," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 6538–6548, 2024.
- [16] K. Kazemi, M. H. Badieli, and H. Kebriaei, "Robust peer-to-peer federated learning with deep reinforcement learning based client selection against data poisoning attacks," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 1, pp. 262–271, 2026.
- [17] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 1126–1135.
- [18] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," *Advances in Neural Information Processing Systems*, vol. 33, pp. 3557–3568, 2020.
- [19] H. A. Tran, C. Ta, and T. X. Tran, "FedImp: Enhancing federated learning convergence with impurity-based weighting," *IEEE Transactions on Artificial Intelligence*, vol. 7, no. 3, pp. 1652–1665, 2026.
- [20] S. Arisdakessian, O. A. Wahab, A. Mourad, and H. Otrouk, "Towards vox populi in federated learning: A fair and inclusive client selection framework," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1997–2011, 2026.
- [21] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 11, pp. 2884–2896, 2018.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2016, pp. 770–778.
- [23] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.